TRABALHO FINAL · TESTE DE SOFTWARE

Vion

Plano de testes de uma plataforma de preparação para o ENEM

Grupo Trab_Final_3

Bianca Silva Vaz · Gabriel Gomes Giani

Lucas Fernandes Fortuna Lourenço · Lucas de Oliveira Faustino

Paulo Henrique Barreto da Rocha Dantas



O que vamos mostrar

01

O sistema

O que é o Vion e como ele foi construído

02

Requisitos

Documento de visão e histórias em BDD

03

Plano de teste

Objetivo, escopo, ferramentas e estratégia

04

Casos de teste

51 casos descritos em três níveis

05

Execução

Resultados, cobertura e evidências

06

Conclusão

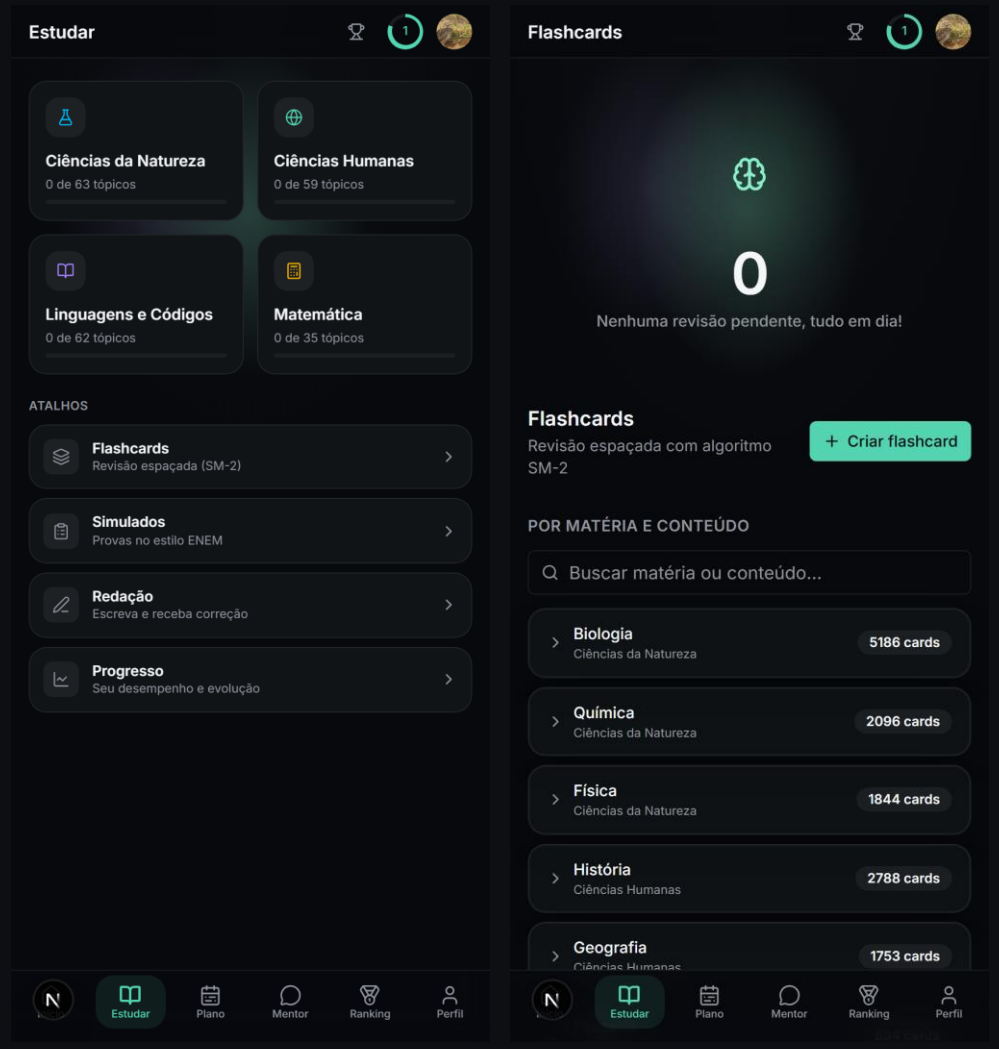
O que aprendemos e o que ficou para depois

O que é o Vion

Tema escolhido: Educação

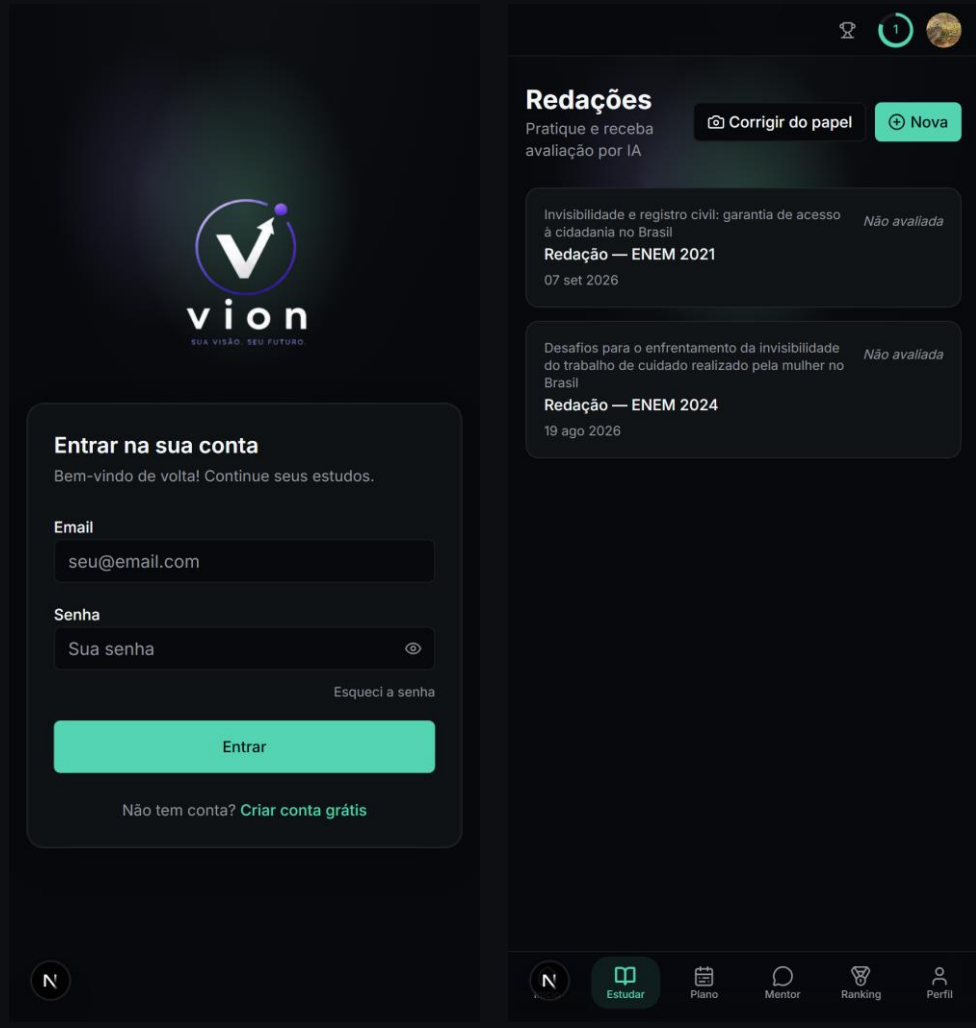
Uma aplicação web que reúne num só lugar as atividades que o estudante de ENEM normalmente espalha entre vários aplicativos.

- Teoria organizada por Área, Disciplina e Tópico
- Flashcards com repetição espaçada (SM-2)
- Questões e simulados no formato da prova
- Redação com correção automática
- Planejador semanal que se ajusta à data do ENEM
- XP, níveis, streak, conquistas e ranking mensal



O sistema cumpre os requisitos obrigatórios

- **Tela de login**
  /login e /register, com recuperação de senha
- **CRUD completo**
  Redação: criar, listar, editar e excluir
- **Interface de usuário**
  Frontend web em Next.js e React
- **Backend com API**
  Route Handlers e Server Actions
- **Banco de dados**
  PostgreSQL acessado via Prisma



Login e o CRUD de redações

Arquitetura

Aplicação full-stack em Next.js 16, com backend no mesmo projeto

Frontend	Next.js 16 · React 19 · Tailwind 4
Backend	Route Handlers · Server Actions
Autenticação	NextAuth 5 · JWT · bcrypt
Validação	Zod 4
Banco	Prisma 6 · PostgreSQL (Neon)

Cerca de 30 modelos

User · UserProfile · Area · Discipline · Topic · Flashcard · FlashcardReview · Question · UserQuestion · UserSimulado · Essay · StudyPlan · Achievement

Documento de visão

Item 4.1 do roteiro

PROBLEMA

Quem estuda para o ENEM espalha a rotina entre vários aplicativos e caderno, e acaba sem enxergar o próprio progresso nem saber onde está errando.

O QUE É

Uma plataforma web que reúne teoria, questões, flashcards, simulados, redação e planejamento, tudo ligado a um histórico de desempenho.

PARA QUEM

Estudantes do ensino médio e vestibulandos que prestam o ENEM, com atenção especial a quem mira cursos concorridos da área de saúde.

DIFERENCIAL

Cada atividade concluída realimenta o sistema: gera XP, mantém a sequência de estudo, alimenta o ranking e ajusta o plano conforme a data da prova se aproxima.

Requisitos em histórias de usuário

Cada história vira critério de aceitação e, daí, caso de teste

HISTÓRIA DE USUÁRIO

Como estudante,

quero registrar minha atividade diária,

para acompanhar minha sequência de estudos.

CRITÉRIOS DE ACEITAÇÃO

- Estudar hoje mantém a sequência
- Estudar ontem ainda mantém
- Um dia inteiro sem estudar zera
- Ler teoria não conta, só prática

CENÁRIO EM BDD

Cenário:	**sequência se mantém quando estudou ontem**
Dado que	minha sequência é de 7 dias
E que	minha última atividade foi ontem
Quando	o sistema calcula a sequência efetiva
Então	o valor apresentado deve ser 7

Este cenário virou o caso de teste CTU-06

Objetivo dos testes

Cobrir os pontos de maior risco, não perseguir 100% de cobertura

1

Regras de gamificação e estudo

Nível, streak, trilha e agendamento de flashcards. É onde está a lógica mais complexa.

2

Autenticação e validação

Porta de entrada do sistema, com regras de segurança como senha forte.

3

CRUD pela API

Ciclo completo da Redação, conferindo se cada operação respeita o dono do recurso.

4

Fluxos ponta a ponta

O caminho real do usuário: cadastrar, entrar, navegar e concluir uma atividade.

Escopo dos testes

SERÁ TESTADO

- Cálculo de nível por XP
- Sequência de dias estudados (streak)
- Escolha da trilha pela data da prova
- Agendamento dos flashcards (SM-2)
- Validação de cadastro e login
- Cadastro, login e recuperação de senha
- Endpoint de perfil e controle de acesso
- CRUD de Redação

NÃO SERÁ TESTADO

- Integrações com IA (não determinísticas)
- Envio real de e-mail (será mockado)
- Tema, animações e paleta de cores
- Vários navegadores e tamanhos de tela
- Desempenho, carga e segurança ofensiva
- Analytics e Ranking em profundidade

Ferramentas

Escolhidas pelo que o roteiro recomenda e pelo que o projeto já usa

Vitest 4

Testes unitários

Já configurado no projeto. API parecida com Jest e entende TypeScript nativo.

Supertest

Testes de API

Cobre os endpoints REST, com o Prisma substituído por mock.

Playwright

Testes E2E

Roda em navegador real e grava print, vídeo e trace de cada falha.

Cobertura V8

Relatórios

Já integrado ao Vitest. Gera relatório em texto e em HTML.

O projeto já tinha o Vitest rodando, com testes das regras de negócio da pasta src/lib.

Estratégia de testes

Três níveis, do mais isolado ao mais próximo do usuário

E2E

10 casos

Fluxos completos pela interface

API

20 casos

Server Actions e Route Handlers, com o Prisma mockado

Unitários

21 casos

Funções puras de src/lib, sem banco e sem rede

Técnicas

Caixa-preta a partir dos requisitos.

Partição de equivalência e análise de valor limite.

Caixa-branca como apoio, guiada pelo relatório de cobertura.

51 casos de teste planejados no total.

Como cada caso foi descrito

Formato exigido pelo roteiro, aplicado nos 51 casos

CTU-02

Fronteira de subida de nível entre 499 e 500

Descrição	Verificar o valor limite na virada do nível 1 para o 2.
Passos	Chamar calcLevel(499) e depois calcLevel(500).
Entrada	499 e 500
Saída esperada	calcLevel(499) = 1 e calcLevel(500) = 2
Critério	Os dois retornos conferem.
Prioridade	Alta

REGRA TESTADA

nível = parte inteira de (XP ÷ 500) + 1, limitado ao máximo de 99

POR QUE ESSE VALOR

499 e 500 são os valores de fronteira: é exatamente ali que o resultado muda. Testar no meio da faixa (por exemplo 250) não revelaria um erro de "maior que" no lugar de "maior ou igual".

Execução dos testes

Unitários e de API rodando no Vitest

66	7	0	2,18s
testes passando	arquivos de teste	falhas	duração total

npm run test

```
$ npm run test

 v src/lib/achievements.test.ts            (11 tests)
 v src/lib/xp.test.ts                      (6 tests)
 v src/lib/trail.test.ts                   (8 tests)
 v src/lib/sm2.test.ts                     (9 tests)
 v src/app/actions/auth.actions.test.ts    (17 tests)
 v src/app/actions/essay.actions.test.ts   (11 tests)
 v src/app/api/profile/route.test.ts       (4 tests)

  Test Files  7 passed (7)
       Tests  66 passed (66)
    Duration  2.18s
```

Saída de npm run test

vitest --reporter=verbose

```
$ npx vitest run --reporter=verbose

  v sm2 > rating 1 (Again) restarts learning              3ms
  v sm2 > rating 2 (Hard) schedules stability * 1.2       1ms
  v sm2 > rating 3 (Good) scales by a difficulty factor   0ms
  v sm2 > rating 4 (Easy) schedules stability * 1.69      0ms
  v sm2 > clamps difficulty to a maximum of 10            0ms
  v sm2 > clamps difficulty to a minimum of 1             0ms
  v sm2 > never schedules less than 1 day                 1ms
  v trail > returns SPRINT when fewer than 90 days remain 0ms
  v trail > has ratios that sum to 1 for every trail      1ms
  v calcLevel > levels up every 500 XP                    1ms
  v calcLevel > caps at level 99                          0ms
```

Detalhe caso a caso

Cobertura de código

Relatório gerado pelo V8, integrado ao Vitest

npm run test:coverage

```
$ npm run test:coverage

 Test Files  4 passed (4)
      Tests  34 passed (34)

 % Coverage report from v8
-------------------|---------|----------|---------|---------
File               | % Stmts | % Branch | % Funcs | % Lines
-------------------|---------|----------|---------|---------
 sm2.ts            |     100 |      100 |     100 |     100
 trail.ts          |     100 |      100 |     100 |     100
 achievements.ts   |   94.23 |    71.42 |   90.62 |   97.87
 league.ts         |   39.39 |       20 |   28.57 |   41.93
 xp.ts             |   26.31 |    10.71 |      50 |   27.77
-------------------|---------|----------|---------|---------
```

ONDE JÁ HÁ TESTES

sm2.ts	100%
trail.ts	100%
achievements.ts	94%

LEITURA DO NÚMERO

A cobertura geral de 22% inclui arquivos que nem entram no escopo, como conexão com o banco e clientes de IA. Nas funções de regra de negócio que já têm teste, ela passa de 94%.

Testes ponta a ponta

Navegador de verdade, sem nada mockado

npx playwright test

```
$ npx playwright test

  Running 4 tests using 1 worker

  ok 1  CTE-01: a tela de cadastro pede nome,
        e-mail e confirmacao de senha          (4.6s)
  ok 2  CTE-02: cadastro com senha fraca
        mostra as regras de senha              (3.0s)
  ok 3  CTE-05: rota protegida sem sessao
        manda pro login                        (2.0s)
  ok 4  CTE-05b: outras rotas protegidas
        tambem exigem sessao                   (16.5s)

  4 passed (29.8s)
```

O QUE OS 12 TESTES COBREM

- Cadastro e regras de senha
- Login certo e login errado
- Rota protegida sem sessão
- Navegação pelo conteúdo
- CRUD de Redação pela tela

EVIDÊNCIA AUTOMÁTICA

Quando um teste falha, o Playwright salva print, vídeo e trace da sessão. Foi assim que descobrimos que o redirecionamento de login apontava para uma porta fixa.

Os 4 testes de controle de acesso rodam sem banco: o middleware bloqueia antes de qualquer consulta.

Defeito encontrado durante o trabalho

Um bug real, achado ao preparar o ambiente de testes

SINTOMA

O build de produção falhava por completo, com a mensagem

"Missing credentials", mesmo sem ninguém usar o Mentor.

CAUSA

O cliente de IA era criado no momento em que o módulo era importado. Como o SDK valida a chave já no construtor, qualquer import do módulo quebrava quando a variável de ambiente não estava disponível, o que acontece na fase de build.

CORREÇÃO

Passamos a criar o cliente só na primeira vez que ele é usado de verdade, e não no import.

LIÇÃO

Efeito colateral em import é armadilha: o código só quebra em um ambiente específico, e não no computador de quem programou. Testar o build num ambiente limpo revelou o que o desenvolvimento local escondia.

Divisão do trabalho no grupo

- **Lucas Fernandes Fortuna Lourenço**
  Coordenação dos testes e relatório
- **Lucas de Oliveira Faustino**
  Testes unitários e acompanhamento da cobertura
- **Gabriel Gomes Giani**
  Testes de API e configuração dos mocks
- **Bianca Silva Vaz**
  Testes E2E e coleta de evidências
- **Paulo Henrique Barreto da Rocha Dantas**
  Ambiente de testes e integração contínua

Todos participam da escrita dos casos e da análise dos resultados.

Status da implementação dos testes

O que já roda e o que ainda falta escrever

Unitários

PRONTO

Vitest

34 testes

- calcLevel, effectiveStreak
- detectTrail, getTrailConfig
- sm2 (agendamento)
- checkAchievements

A suíte roda em 1,85s e passa inteira. São 34 testes, porque alguns casos do plano viraram mais de um teste no código.

API

A FAZER

Supertest

0 de 20 casos

- Mock do Prisma Client
- register, login, reset de senha
- GET /api/profile
- CRUD de Redação

Os 20 casos já estão descritos no relatório. Falta escrever o código e configurar o mock do banco.

E2E

A FAZER

Playwright

0 de 10 casos

- Instalar e configurar
- Banco de teste com seed
- Cadastro, login e acesso
- CRUD pela interface

O Playwright já foi usado para gerar os prints deste trabalho, mas ainda não como suíte de teste automatizada.

78 testes automatizados no total: 66 no Vitest e 12 no Playwright.

Entregas pedidas no roteiro

Onde cada item está e o que ainda falta

4.1	**Documento de visão**	Apresentado neste slide deck	OK
4.1	**Histórias com critérios e BDD**	Uma história completa; as demais em elaboração	PARCIAL
4.2	**Plano de teste**	Objetivo, escopo, ferramentas, estratégia e papéis	OK
4.2	**Casos de teste**	51 casos com ID, passos, entrada, saída e critério	OK
4.3	**Implementação nos frameworks**	Vitest para unitário e API, Playwright para E2E	OK
4.4	**Execução e evidências**	78 testes rodando, cobertura e dois defeitos achados	OK
4.5	**Relatório final**	Documento entregue com introdução, plano e casos	OK
4.5	**Link do repositório**	github.com/LucasF89k/enem-hub	OK

Falta completar as demais histórias de usuário em BDD.

Conclusão e lições aprendidas

1 Testar valor de fronteira vale mais

Os erros aparecem na virada, não no meio da faixa. Testar 499 e 500 acha o que testar 250 nunca acharia.

2 Escopo bem definido protege o trabalho

Dizer o que NÃO seria testado, como as integrações de IA, evitou perder tempo com teste instável que falha por motivo alheio ao código.

3 Mock separa o que se quer testar

Trocar o banco por mock deixou a gente testar "e-mail já existe" e "token expirado" sempre do mesmo jeito, sem depender de dado real.

4 Ambiente limpo revela bug escondido

O defeito do build só apareceu fora da máquina de quem programou. Rodar em ambiente limpo é parte de testar.

Situação atual e próximos passos

Feito	**Sistema**	Login, CRUD, API e banco em produção
Feito	**Plano de teste**	Escopo, ferramentas, estratégia e papéis
Feito	**51 casos descritos**	21 unitários, 20 de API e 10 E2E
Parcial	**Implementação**	34 testes unitários rodando; falta API e E2E
A fazer	**Evidências completas**	Rodar a suíte inteira e registrar tudo

Critério de saída: todos os casos executados · todos os de prioridade alta aprovados · cobertura mínima de 80% nas funções críticas

Obrigado

ACESSE O SISTEMA

enem-hub.vercel.app

Grupo Trab_Final_3 · Teste de Software

Aponte a câmera

